

Лекция 1.

Введение в численные методы и основы теории погрешностей

1 Основы теории погрешностей

1.1 Типы погрешностей

1. Погрешность задачи (Неустраняемая погрешность)

Погрешность задачи возникает еще до начала каких-либо вычислений на компьютере. Она обусловлена двумя факторами:

1. **Неполнота математической модели:** невозможно учесть абсолютно все физические свойства объекта (например, при расчете движения тела пренебрегают сопротивлением воздуха или формой Земли).
2. **Неточность входных данных:** начальные параметры почти всегда получают экспериментально с помощью приборов, которые имеют свою конструктивную погрешность измерения.

Эта погрешность является **неустранимой (безусловной)**. Никакой идеальный алгоритм или мощный процессор не могут уменьшить эту ошибку, так как она заложена в математическую постановку изначально.

ПРИМЕР: РАСЧЕТ ТРАЕКТОРИИ СПУТНИКА ИЛИ ВЫСОТНОГО ЗОНДА

Нам необходимо рассчитать высоту полета исследовательского аппарата, запущенного вертикально вверх.

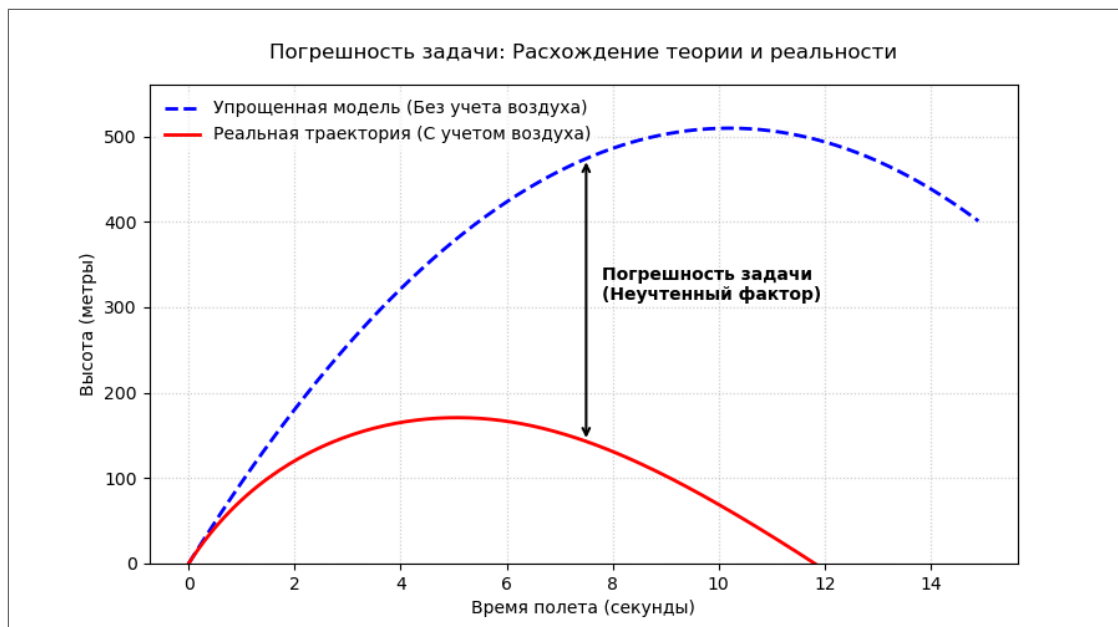
- **Что мы учли (Упрощенная модель):** Равноускоренное движение в вакууме по школьной формуле:

$$h(t) = h_0 + v_0 t - \frac{gt^2}{2}$$

Здесь ускорение свободного падения g принято за константу (9.81 м/с^2).

- **Что мы НЕ учли (Источники погрешности задачи):**
 1. **Соппротивление воздуха:** На высоких скоростях плотные слои атмосферы создают колоссальное торможение.
 2. **Неоднородность гравитационного поля:** По мере удаления от Земли сила тяжести ослабевает (величина g падает), а Земля из-за своей формы (геоид) притягивает объекты неравномерно.

- **Итог:** На первых секундах полета формула работает неплохо. Но при долгосрочном прогнозе неучтенное сопротивление воздуха приведет к тому, что реальный аппарат окажется на километры ниже, чем предсказала наша «чистая» математическая модель. Вероятностные методы здесь бессильны, так как ошибка формулы строго направлена в одну сторону.



2. Погрешность метода

Погрешность метода возникает из-за того, что реальные математические операции часто бывают непрерывными, а компьютер может выполнять только конечное число дискретных шагов; сложные модели аппроксимируются простыми линейными моделями и т.п. Нам приходится урезать бесконечные формулы или заменять сложные кривые простыми отрезками.

В отличие от погрешности задачи, погрешность метода является **устранимой**. Математик может уменьшить её до сколь угодно малого значения, увеличивая точность алгоритма (например, уменьшая шаг расчетов или добавляя больше элементов в вычисления). Однако за это приходится платить временем работы процессора.

Пример: Численное интегрирование (Метод прямоугольников)

Нам нужно найти площадь под кривой (определенный интеграл) функции $f(x) = x^2$ на отрезке от 0 до 2. Точный аналитический ответ: **2.666...**

- **Как считает метод:** Компьютер разбивает отрезок на N равных частей шириной h (шаг сетки) и строит на них прямоугольники. Сумма площадей этих прямоугольников принимается за ответ.
- **Где возникает ошибка:** Между вершинами прямоугольников и реальной плавной кривой остаются пустые «кусочки» (неучтенная площадь). Сумма этих кусочков и есть погрешность метода.
- **Как её устранить:** Если разбить отрезок не на 4 части, а на 1000, прямоугольники станут очень узкими, «кусочки» ошибок уменьшатся, и ответ станет максимально близким к точному (Теорема сумм Дарбу) .

3. Погрешность вычислений (Погрешность округления)

Погрешность вычислений возникает непосредственно в процессе выполнения арифметических операций на компьютере. Её главная причина — ограниченная разрядная сетка. Компьютер оперирует числами в двоичной системе (стандарт IEEE 754), из-за чего многие простые десятичные дроби (например, **0.1** или **0.2**) превращаются в бесконечные периодические дроби и вынужденно округляются при записи в память.

При выполнении миллионов последовательных операций эти микро-округления могут накапливаться, приводить к потере значащих цифр и существенно исказить итоговый результат.

В рамках этого типа погрешности выделяют две фундаментальные задачи теории ошибок:

1. **Прямая задача:** Оценка погрешности результата функции по известным погрешностям её аргументов.
 - *Математическая оценка:* Для дифференцируемой функции $y = f(x_1, x_2, \dots, x_n)$ предельная абсолютная погрешность вычисляется через частные производные:
 2. **Обратная задача:** Нахождение допустимых погрешностей аргументов, которые гарантированно обеспечат заданную точность финального результата.
-

Пример: Эффект накопления в цикле

Представьте алгоритм, который в цикле складывает число 0.1 ровно 10 000 раз.

- **Ожидаемый математический результат: 1000.0**
- **Реальный результат на ПК:** Компьютер выдаст число вида 999.9028930664062 для типа float32.

Этот крошечный зазор — следствие того, что на каждом шаге цикла процессор совершал вынужденное округление двоичного представления числа. В масштабах больших вычислений (например, при решении огромных систем линейных уравнений или дифференциальных уравнений на больших сетках) погрешность округления может полностью уничтожить точность метода, если алгоритм неустойчив.

In [26]:

```
import numpy as np

step = np.float32(0.1)

expected = 1000.0

result = np.float32(0.0)

for _ in range(10000):
    result += step

abs_error = abs(expected - result)
rel_error = (abs_error / expected) * 100
```

Ожидаемый точный результат: 1000.0
Реальный результат на ЭВМ: 999.9028930664062
Абсолютная ошибка округления: 0.09710693359375
Относительная ошибка: 0.00971%

1.2 Методы оценки погрешностей

Пусть нам известна некоторая физическая или математическая величина. Обозначим её компоненты в соответствии с принятой академической терминологией:

- X — **точное значение** величины (истинное значение, которое в реальных прикладных задачах чаще всего остается абстракцией);
 - x — **приближенное значение** этой величины (число, полученное в результате физического измерения, округления или промежуточного расчета на компьютере).
-

Абсолютная погрешность

ОПРЕДЕЛЕНИЕ 1.2.1 (ИСТИННАЯ АБСОЛЮТНАЯ ПОГРЕШНОСТЬ)

Абсолютной погрешностью приближенного числа x называется разность между точным значением X и приближенным x :

$$\Delta_x = |X - x|$$

ОПРЕДЕЛЕНИЕ 1.2.2 (ПРЕДЕЛЬНАЯ АБСОЛЮТНАЯ ПОГРЕШНОСТЬ)

Поскольку точное значение X обычно неизвестно, определить истинную погрешность Δ_x невозможно. На практике оперируют понятием верхней границы.

Предельной абсолютной погрешностью приближенного числа x (обозначается как $\Delta(x)$) называется любое неотрицательное число, заведомо удовлетворяющее неравенству:

$$|\Delta_x| = |X - x| \leq \Delta(x)$$

Из этого определения следует, что истинное значение величины X гарантированно локализовано в интервале:

$$X \in [x - \Delta(x), x + \Delta(x)] \quad \text{ИЛИ} \quad X = x \pm \Delta(x)$$

Относительная погрешность

Абсолютная погрешность не всегда отражает реальное качество проведенного расчета или измерения. Для качественной оценки точности вводится относительная мера, которая привязывает величину ошибки к масштабу самого числа.

ОПРЕДЕЛЕНИЕ 1.2.3 (ИСТИННАЯ ОТНОСИТЕЛЬНАЯ ПОГРЕШНОСТЬ)

Относительной погрешностью приближенного числа x называется отношение его истинной абсолютной погрешности к модулю точного значения X :

$$\delta_x = \frac{\Delta_x}{|X|}$$

ОПРЕДЕЛЕНИЕ 1.2.4 (ПРЕДЕЛЬНАЯ ОТНОСИТЕЛЬНАЯ ПОГРЕШНОСТЬ)

Предельной относительной погрешностью приближенного числа x (обозначается как $\delta(x)$) называется отношение его предельной абсолютной погрешности к модулю самого числа:

$$\delta(x) = \frac{\Delta(x)}{|x|}$$

Примечание: Предельная относительная погрешность $\delta(x)$ является безразмерной величиной и часто выражается в процентах ($\delta(x) \cdot 100\%$). По Вержбицкому, в знаменателе теоретического определения строго должно стоять точное значение $|X|$, однако ввиду близости $x \approx X$ в практических вычислениях используют известное приближение x .

▷ **Замечание о терминологии:** Так как истинные погрешности нам не известны, то далее в тексте под символами $\Delta(x)$ и $\delta(x)$ мы почти всегда будем подразумевать и называть их просто **абсолютной** и **относительной** погрешностью, опуская для краткости слово «предельная».

Аналитический способ учёта погрешностей

Пусть задана дифференцируемая функция нескольких переменных $y = f(x_1, x_2, \dots, x_n)$.

Предположим, что точные значения аргументов x_i нам неизвестны, но заданы их приближенные значения x_i^* ($i = 1, 2, \dots, n$). Для каждого аргумента определены:

- **Предельная абсолютная погрешность** Δ_{x_i}
- **Предельная относительная погрешность** δ_{x_i}

Требуется найти предельную абсолютную (Δ_y) и предельную относительную (δ_y) погрешности приближенного значения функции $y^* = f(x_1^*, x_2^*, \dots, x_n^*)$.

ОБЩАЯ ФОРМУЛА АБСОЛЮТНОЙ И ОТНОСИТЕЛЬНОЙ ПОГРЕШНОСТИ

$$\Delta_y = \sum_{i=1}^n \left| \frac{\partial f(x_1^*, x_2^*, \dots, x_n^*)}{\partial x_i} \right| \Delta_{x_i}$$
$$\delta_y = \sum_{i=1}^n \left| \frac{\partial(\ln f(x^*))}{\partial x_i} \right| \Delta_{x_i} = \sum_{i=1}^n \left| \frac{\partial f(x^*)}{\partial x_i} \cdot \frac{x_i^*}{f(x^*)} \right| \delta_{x_i}$$

Сводная таблица погрешностей классических функций

Функция (y)	Предельная абсолютная (Δ_y)	Предельная относительная (δ_y)
$x_1 + x_2$	$\Delta_{x_1} + \Delta_{x_2}$	$\frac{\Delta_{x_1} + \Delta_{x_2}}{ x_1^* + x_2^* }$
$x_1 - x_2$	$\Delta_{x_1} + \Delta_{x_2}$	$\frac{\Delta_{x_1} + \Delta_{x_2}}{ x_1^* - x_2^* }$
$x_1 \cdot x_2$	$ x_2^* \Delta_{x_1} + x_1^* \Delta_{x_2}$	$\delta_{x_1} + \delta_{x_2}$
x_1 / x_2	$\frac{ x_2^* \Delta_{x_1} + x_1^* \Delta_{x_2}}{(x_2^*)^2}$	$\delta_{x_1} + \delta_{x_2}$

▷ Доказательство на лекции))

1.3 Общие правила округления чисел. Правила Крылова

ОПРЕДЕЛЕНИЕ 1.3.1 (ЗНАЧАЩИЕ ЦИФРЫ)

Значащими цифрами приближённого десятичного числа называются все цифры в его записи, начиная с первой ненулевой слева и до последней записанной цифры справа (включая нули между ними и в конце).

ОПРЕДЕЛЕНИЕ 1.3.2 (ВЕРНЫЕ ЦИФРЫ)

Цифра α_k числа α **верна** в узком смысле, если $\Delta_a \leq 0.5 \cdot 10^k$

ПРИМЕР, ГДЕ **ВСЕ** ЗНАЧАЩИЕ ЦИФРЫ ВЕРНЫ

Пусть $a = 3.1416$ — приближение к $A = 3.14159$.

Тогда $\Delta_a = |3.14159 - 3.1416| = 0.00001$.

Проверим каждый разряд (от старшего к младшему):

Цифра	Разряд k	Условие $\Delta_a \leq 0.5 \cdot 10^k$	Верна?
3	0	$0.00001 \leq 0.5$ — да	Да
1	-1	$0.00001 \leq 0.05$ — да	Да
4	-2	$0.00001 \leq 0.005$ — да	Да
1	-3	$0.00001 \leq 0.0005$ — да	Да
6	-4	$0.00001 \leq 0.00005$ — да	Да

Вывод: все 5 значащих цифр числа **3.1416** являются верными (в узком смысле).

ПРИМЕР, ГДЕ **НЕ ВСЕ** ЗНАЧАЩИЕ ЦИФРЫ ВЕРНЫ

Пусть $a = 3.14$ — приближение к $A = 3.146$.

Тогда $\Delta_a = |3.146 - 3.14| = 0.006$.

Значащие цифры числа **3.14**: **3, 1, 4** (всего 3).

Проверка верности:

Цифра	Разряд k	Условие $\Delta_a \leq 0.5 \cdot 10^k$	Верна?
3	0	$0.006 \leq 0.5$ — да	Да
1	-1	$0.006 \leq 0.05$ — да	Да
4	-2	$0.006 \leq 0.005$ — нет ($0.006 > 0.005$)	Нет

Вывод: цифры **3** и **1** — верные, а цифра **4** — **не верна**, хотя она значащая.

Таким образом, число **3.14** имеет только **2 верные значащие цифры** (в узком смысле).

ЕЩЁ ПРИМЕР С НУЛЯМИ

$a = 0.02030$, $A = 0.02035$, тогда $\Delta_a = 0.00005$.

Значащие цифры: 2, 0, 3, 0 (ведущие нули не считаем).

Проверка:

- 2 (разряд 10^{-2}): $0.00005 \leq 0.5 \cdot 10^{-2} = 0.005$ — верна
- 0 (разряд 10^{-3}): $0.00005 \leq 0.5 \cdot 10^{-3} = 0.0005$ — верна
- 3 (разряд 10^{-4}): $0.00005 \leq 0.5 \cdot 10^{-4} = 0.00005$ —
верна (равенство допускается)
- 0 (разряд 10^{-5}): $0.00005 \leq 0.5 \cdot 10^{-5} = 0.000005$ — **не
верна**

Вывод: первые три значащие цифры (2, 0, 3) верны, последний ноль (в разряде 10^{-5}) — не верен. При этом все четыре цифры значащие.

Правила Крылова для приближённых вычислений

Академик А.Н. Крылов сформулировал **три практических правила** записи и обработки приближённых чисел, которые позволяют сохранять разумную точность без излишней перегрузки цифрами.



Основной принцип Крылова

Приближённое число следует записывать так, чтобы все его значащие цифры, кроме последней, были верными, и лишь последняя цифра была сомнительной, но не более чем на одну единицу её разряда.

Математически это означает, что если последняя цифра стоит в разряде с весом 10^k , то абсолютная погрешность числа удовлетворяет условию:

$$\Delta_a \leq 1 \cdot 10^k$$

Это соответствует **широкому смыслу верной цифры** и даёт некоторый «запас» при вычислениях.



Три ПРАВИЛА КРЫЛОВА

Правило 1. Сложение и вычитание

*При сложении и вычитании приближённых чисел в **окончательном результате** сохраняют **столько десятичных знаков** (цифр после запятой), сколько их в числе с **наименьшим** числом десятичных знаков среди всех слагаемых.*

Обоснование: абсолютная погрешность суммы равна сумме абсолютных погрешностей слагаемых, поэтому точность результата определяется наименее точным слагаемым.

Примеры:

Выражение	Промежуточный результат	Окончательная запись	Пояснение
$127.42 + 67.3 + 0.12$	194.84	194.8	Оставляем 1 знак после запятой (как у 67.3)
$3.1415 + 2.71$	5.8515	5.85	Оставляем 2 знака после запятой (как у 2.71)
$1000.0 + 0.123$	1000.123	1000.1	Один знак после запятой (как у 1000.0)

ПРАВИЛО 2. УМНОЖЕНИЕ И ДЕЛЕНИЕ

*При умножении и делении приближённых чисел в **окончательном результате** сохраняют **столько значащих цифр**, сколько их в числе с **наименьшим** числом значащих цифр среди всех сомножителей (делимого и делителя).*

Обоснование: относительная погрешность произведения (частного) равна сумме относительных погрешностей сомножителей, поэтому число верных значащих цифр определяется наименее точным множителем.

Примеры:

Выражение	Промежуточный результат	Окончательная запись	Пояснение
3.14×2.7	8.478	8.5	2 значащие цифры (как у 2.7)
$125.6 \div 4.0$	31.4	3.1×10^1 или 31	2 значащие цифры (как у 4.0)
0.0034×12.56	0.042704	0.043	2 значащие цифры (как у 0.0034)

ПРАВИЛО 3. ПРОМЕЖУТОЧНЫЕ РЕЗУЛЬТАТЫ

*Чтобы избежать накопления погрешности округления, все **промежуточные результаты** следует записывать с **1–2 запасными знаками** (цифрами) по сравнению с тем, сколько требуется по правилам 1 и 2. Окончательное округление до нужного числа знаков выполняется только в самом конце вычислений.*

Обоснование: каждое промежуточное округление вносит дополнительную погрешность, которая может накапливаться и исказить последние верные цифры окончательного результата.

Пример:

Пусть требуется вычислить $S = 3.14 \times 2.718 \times 1.41$.

- По правилу 2 в ответе нужно оставить 3 значащие цифры (так как **3.14** — 3 значащие, **2.718** — 4, **1.41** — 3; минимальное число — 3).
- В промежуточных вычислениях берём на 2 знака больше:
 - $3.14 \times 2.718 = 8.53452$ (оставляем 5 значащих цифр: **8.5345**)
 - $8.5345 \times 1.41 = 12.033645$ (оставляем 5 значащих: **12.034**)
- Окончательный ответ округляем до 3 значащих цифр: **12.0**.

СВЯЗЬ ПРАВИЛ КРЫЛОВА С ПОГРЕШНОСТЯМИ

Каждое правило прямо следует из оценок погрешностей:

Операция	Погрешность результата	Следствие для записи
$S = a + b$	$\Delta_S \leq \Delta_a + \Delta_b$	Ориентируемся на слагаемое с бóльшим Δ (меньше знаков после запятой)
$P = a \cdot b$	$\delta_P \leq \delta_a + \delta_b$	Ориентируемся на множитель с бóльшей относительной погрешностью (меньше значащих цифр)
Промежуточные округления	Вносят дополнительную погрешность на каждом шаге	Держим запасные цифры, чтобы финальная погрешность не превысила допустимую

 ВАЖНЫЕ ЗАМЕЧАНИЯ

1. **Правила Крылова — не жёсткие теоремы, а практические рекомендации.** Они дают разумный компромисс между точностью и удобством записи.
2. **Для максимально точных расчётов** используют **узкий смысл верных цифр** ($\Delta \leq 0.5 \cdot 10^k$), а не широкий (по Крылову). В таких случаях в промежуточных вычислениях берут ещё больше запасных знаков.
3. **Если в числе много нулей в конце** — их нужно выписывать, если они верны. Например, запись **1200** не показывает точность, а запись **1200.0** (с точностью до десятых) — показывает. Это соответствует принципу Крылова.
4. **При реализации численных алгоритмов** (например, численном интегрировании, решении дифференциальных уравнений или обработке больших массивов данных) нам приходится складывать **миллионы** чисел.

ПРИМЕР: КАТАСТРОФИЧЕСКИЙ ВЗРЫВ ПОГРЕШНОСТИ ПРИ ВЫЧИТАНИИ

Покажем на простом численном примере, как операция вычитания близких чисел мгновенно уничтожает точность результата, даже если исходные данные были измерены с высокой точностью.

Пусть в ходе эксперимента измеряются две близкие величины x_1 и x_2 . Приборы зафиксировали следующие приближенные значения и их предельные абсолютные погрешности:

- $x_1^* = 10.05 \pm 0.02$
- $x_2^* = 10.01 \pm 0.02$

1. ОЦЕНКА ТОЧНОСТИ ИСХОДНЫХ ДАННЫХ

Оценим относительные погрешности аргументов. Оба числа измерены качественно, их относительные ошибки ничтожно малы:

$$\bullet \delta_{x_1} = \frac{\Delta_{x_1}}{|x_1^*|} = \frac{0.02}{10.05} \approx 0.002 \implies \mathbf{0.2\%}$$

$$\bullet \delta_{x_2} = \frac{\Delta_{x_2}}{|x_2^*|} = \frac{0.02}{10.01} \approx 0.002 \implies \mathbf{0.2\%}$$

2. ВЫЧИСЛЕНИЕ РАЗНОСТИ $y = x_1 - x_2$

Найдем приближенное значение разности аргументов:

$$y^* = x_1^* - x_2^* = 10.05 - 10.01 = \mathbf{0.04}$$

3. РАСЧЕТ ПОГРЕШНОСТИ РЕЗУЛЬТАТА

Согласно правилу накопления ошибок при вычитании, предельные абсолютные погрешности аргументов складываются:

$$\Delta_y = \Delta_{x_1} + \Delta_{x_2} = 0.02 + 0.02 = \mathbf{0.04}$$

Теперь рассчитаем предельную относительную погрешность полученного результата δ_y :

$$\delta_y = \frac{\Delta_y}{|y^*|} = \frac{0.04}{0.04} = 1.0 \implies \mathbf{100\%}$$

1.4 Стандарт IEEE 754

IEEE — Institute of Electrical and Electronics Engineers
(Институт инженеров электротехники и электроники).
Международная некоммерческая ассоциация специалистов
в области техники, являющаяся мировым лидером в
разработке стандартов по радиоэлектронике,
электротехнике и вычислительным системам.

IEEE 754 (IEC 60559) — стандарт представления чисел с плавающей точкой

IEEE 754 (также известен как **IEC 60559**) — международный стандарт, определяющий форматы представления чисел с плавающей точкой в компьютерных системах. Он регламентирует:

- двоичное и десятичное представление,
 - правила округления,
 - обработку исключительных ситуаций (переполнение, деление на ноль и т.д.),
 - специальные значения ($\pm\infty$, NaN, ± 0).
-



ОСНОВНЫЕ ФОРМАТЫ (BINARY) — ИСПОЛЬЗУЕМЫЕ ТИПЫ

Тип в языках (C/C++, Java)	Стандартное имя	Бит всего	Знак (S)	Экспонента (E)	Мантисса (M)	bias	П д
float	binary32	32	1	8	23	127	=
double	binary64	64	1	11	52	1023	=
long double (x86)	binary80 (extended)	80	1	15	64	16383	=

В большинстве вычислений используют double (binary64) как стандарт для научных расчётов.

 СТРУКТУРА ЧИСЛА (НА ПРИМЕРЕ FLOAT32)

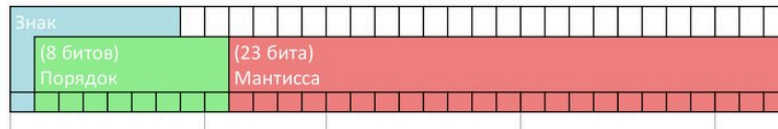
Число с плавающей точкой в нормализованном виде хранится как:

$$x = (-1)^S \cdot 2^{E-\text{bias}} \cdot (1 + M)$$

где:

- S — знаковый бит (0 — плюс, 1 — минус),
 - E — смещённая экспонента (целое без знака),
 - bias — константа смещения (для float — 127, для double — 1023),
 - M — дробная часть мантииссы (23 бита для float), причём старший бит (единица) не хранится, так как он всегда равен 1 для нормализованных чисел (это **скрытый бит**).
-

Представление float – стандарт IEEE 754



Порядок	Мантисса 0	Мантисса != 0	Формула
0x00	0 и -0	Денормализов. числа	$(-1)^{\text{знак}} \cdot 2^{\text{порядок}-126} \cdot (0.\text{мантисса})_{(2)}$
0x01 ... 0xfe	Нормализованные числа		$(-1)^{\text{знак}} \cdot 2^{\text{порядок}-127} \cdot (1.\text{мантисса})_{(2)}$
0xff	$+\infty$ или $-\infty$	NaN	



СПЕЦИАЛЬНЫЕ ЗНАЧЕНИЯ: нули, БЕСКОНЕЧНОСТИ, NaN

Стандарт выделяет особые комбинации бит, которые не являются обычными числами:

Случай	Экспонента (E)	Мантисса (M)	Зачем это нужно
Нормализованное число	$1 \leq E \leq 2^n - 2$	любое	Обычные числа, скрытый бит = 1
Ноль (+0, −0)	0	0	Позволяет сохранить знак при стремлении к нулю (например, результат переполнения снизу). −0 и +0 сравниваются как равные, но знак может влиять на некоторые функции (например, $1/(-0) = -\infty$).

Случай	Экспонента (E)	Мантисса (M)	Зачем это нужно
Денормализованное число (субнормальное)	0	$\neq 0$	Позволяет плавно переходить к нулю, предотвращая резкий скачок точности (gradual underflow). Без них наименьшее нормализованное число 2^{-126} сменялось бы сразу нулём, что вызвало бы потерю точности.
Бесконечность ($+\infty$, $-\infty$)	все биты = 1	0	Возникает при переполнении (деление на ноль, результат больше максимального). Позволяет продолжать вычисления и корректно обрабатывать пределы.
NaN (Not a Number)	все биты = 1	$\neq 0$	Результат неопределённой операции ($0/0$, $\infty - \infty$). Используется для сигнализации об ошибках без

Случай	Экспонента (E)	Мантисса (M)	Зачем это нужно
			остановки программы.



РЕЖИМЫ ОКРУГЛЕНИЯ

1. Округление к ближайшему (round to nearest, ties to even)

Это **режим по умолчанию**. Выбирается ближайшее представимое число. Если результат ровно посередине между двумя соседними числами, выбирается то, у которого **последний бит мантиссы равен 0** (чётное). Это банковское округление — оно не даёт систематического смещения вверх.

Пример (для десятичного представления, чтобы было понятно):

- Округлить до целых: $2.5 \rightarrow 2$ (чётное), $3.5 \rightarrow 4$ (чётное), $2.4 \rightarrow 2$, $2.6 \rightarrow 3$.
-

2. Округление к $+\infty$ (round toward $+\infty$)

Всегда округляет вверх (к большему числу). Для отрицательных чисел это означает уменьшение модуля (например, $-2.1 \rightarrow -2$).

3. Округление к $-\infty$ (round toward $-\infty$)

Всегда округляет вниз (к меньшему числу). Для отрицательных чисел это увеличение модуля (например, $-2.1 \rightarrow -3$).

4. Округление к нулю (round toward zero)

Простое отбрасывание лишних разрядов (усечение). Для положительных работает как вниз, для отрицательных — как вверх (например, $2.9 \rightarrow 2$, $-2.9 \rightarrow -2$).

 МАШИННЫЙ ЭПСИЛОН

Машинный эпсилон ($\varepsilon_{\text{mach}}$) — это наименьшее положительное число $\varepsilon > 0$, такое что при сложении с **1** в данном формате с плавающей точкой результат **отличается от 1**:

$$\varepsilon_{\text{mach}} = \min\{\varepsilon > 0 \mid 1 + \varepsilon \neq 1\}$$

В нормализованных двоичных форматах он вычисляется как:

$$\varepsilon_{\text{mach}} = 2^{-p}$$

где p — число бит мантиссы. Для:

- **binary32 (float):** $p = 23$.
 - **binary64 (double):** $p = 52$.
-

In [28]:

```
one_32 = np.float32(1.0)
eps_32 = np.float32(2** -23)
half_eps_32 = np.float32(eps_32 / 2)
result = one_32 + half_eps_32
result2 = one_32 + eps_32
```

Машинное эpsilon для float32: 1.1920928955078125e-07

Половина эpsilon для float32: 5.960464477539063e-08

Результат сложения (1.0 + half_eps): 1.0

Результат сложения (1.0 + eps_32): 1.0000001192092896

Пример с денормализованными числами

In [29]:

```
import struct
import numpy as np

def print_float32_bits(val):
    """Функция принимает np.float32 и печатает его побитовую структуру"""
    [binary_integer] = struct.unpack("I", struct.pack("f", val))
    b_str = f"{binary_integer:032b}"

    sign = b_str[0]
    exponent = b_str[1:9]
    mantissa = b_str[9:]

    print(f"Число: {val}")
    print(f"Биты:  Знак:[{sign}]  Экспонента:[{exponent}]  Мантисса:[{mantissa}]"
          print("-" * 80))

norm_32 = np.float32(2** -126)
denorm_32 = np.float32(2** -126 / 2)
abs_min_denorm_32 = np.finfo(np.float32).smallest_subnormal
```

Число: 1.1754943508222875e-38

Биты: Знак:[0] Экспонента:[00000001] Мантисса:[0000000000000000
00000000]

Число: 5.877471754111438e-39

Биты: Знак:[0] Экспонента:[00000000] Мантисса:[1000000000000000
00000000]

Число: 1.401298464324817e-45

Биты: Знак:[0] Экспонента:[00000000] Мантисса:[0000000000000000
00000001]

Пример работы GRS битов

In [33]:

```
import numpy as np

x1 = np.float32(1.0)

x2 = np.float32(0.5 + 2** -24)

real_result = x1 - x2
no_grs_result = np.float32(1.0 - 0.5)
diff = no_grs_result - real_result
```

Число: 1.0

Биты: Знак:[0] Экспонента:[01111111] Мантисса:[0000000000000000
00000000]

Число: 0.5000000596046448

Биты: Знак:[0] Экспонента:[01111110] Мантисса:[0000000000000000
00000001]

Результат с GRS (Реальный процессор):

Число: 0.4999999403953552

Биты: Знак:[0] Экспонента:[01111101] Мантисса:[1111111111111111
11111110]

Результат БЕЗ GRS (Теоретический):

Число: 0.5

Биты: Знак:[0] Экспонента:[01111110] Мантисса:[0000000000000000
00000000]

Разница, которую спасли GRS-биты: 5.960464477539063e-08 (это
в точности 2^{*-24})

1.5 Примеры неустойчивых задач

Пример 1: Анализ численной неустойчивости уравнения

$$(x - a)^n = \varepsilon$$

1. ИСХОДНЫЕ ПАРАМЕТРЫ

Зафиксируем конкретные жесткие параметры для демонстрации эффекта:

- **Параметр смещения (a):** 5 (истинный корень)
 - **Степень многочлена (n):** 50 (высокая степень)
 - **Малое возмущение (ε):** 10^{-50} (число, близкое к нулю)
-

2. ИДЕАЛЬНЫЙ СЛУЧАЙ (БЕЗ ВОЗМУЩЕНИЯ)

Если правая часть строго равна нулю ($\varepsilon = 0$), уравнение имеет вид:

$$(x - 5)^{50} = 0$$

Результат:

- Существует **один** единственный корень кратности 50.
 - Точный ответ: $x_{\text{точный}} = 5$.
-

3. РЕАЛЬНЫЙ СЛУЧАЙ (СВЕРХМАЛОЕ ε)

Теперь вносим то самое маленькое изменение $\varepsilon = 10^{-50}$:

$$(x - 5)^{50} = 10^{-50}$$

Из-за высокой степени $n = 50$ один кратный корень мгновенно рассыпается в комплексной плоскости на 50 отдельных изолированных корней.

4. РЕШЕНИЕ ЗАДАЧИ С ВОЗМУЩЁННЫМ КОЭФФИЦИЕНТОМ

Чтобы найти новые корни, извлечем корень 50-й степени. Используем комплексные числа и формулу Муавра:

$$x_k - 5 = \sqrt[50]{10^{-50}} \cdot e^{\frac{2\pi i k}{50}}, \quad k = 0, 1, \dots, 49$$

Вычислим модуль (величину) отклонения новых корней от истинного значения 5:

$$\Delta x = \sqrt[50]{10^{-50}} = 10^{-1} = 0.1$$

Что произошло геометрически:

Вместо одной точки $x = 5$ на окружности радиуса 0.1 выстроилось 50 комплексных корней. Некоторые из них остались вещественными:

- $x_1 = 5 + 0.1 = 5.1$
- $x_2 = 5 - 0.1 = 4.9$

Пример 2: Численная неустойчивость СЛУ (Плохая обусловленность)

1. ИСХОДНАЯ СИСТЕМА (ИДЕАЛЬНЫЙ СЛУЧАЙ)

Рассмотрим систему двух уравнений с двумя неизвестными:

$$\begin{cases} x + 10y = 11 \\ 100x + 1001y = 1101 \end{cases}$$

Точное решение:

Умножим первое уравнение на 100 и вычтем его из второго:

$$(100x + 1001y) - (100x + 1000y) = 1101 - 1100$$

$$1y = 1 \implies y = 1$$

Подставим $y = 1$ в первое уравнение:

$$x + 10(1) = 11 \implies x = 1.$$

Точный ответ:

$$\begin{cases} x = 1 \\ y = 1 \end{cases}$$

2. ВОЗМУЩЕННАЯ СИСТЕМА (ВНОСИМ ПОГРЕШНОСТЬ)

Предположим, что из-за ошибки округления при вводе данных свободный член второго уравнения изменился всего на **~0.09%** (вместо 1101 компьютер записал 1102).

Новая система принимает вид:

$$\begin{cases} x + 10y = 11 \\ 100x + 1001y = 1102 \end{cases}$$

3. РАСЧЕТ НОВОГО (ИСКАЖЕННОГО) РЕШЕНИЯ

Решим измененную систему тем же методом:

1. Умножим первое уравнение на 100:

$$100x + 1000y = 1100$$

2. Вычтем полученное уравнение из обновленного второго уравнения:

$$(100x + 1001y) - (100x + 1000y) = 1102 - 1100$$

$$1y = 2 \implies y = 2$$

3. Подставим $y = 2$ обратно в первое уравнение:

$$x + 10(2) = 11 \implies x + 20 = 11 \implies x = -9$$

Новый ответ:

$$\begin{cases} x = -9 \\ y = 2 \end{cases}$$

4. СРАВНЕНИЕ МАСШТАБА ОШИБОК

Посмотрим, как ничтожный сдвиг на входе полностью перевернул ответ:

Параметр	Исходное значение	Измененное значение	Абсолютный сдвиг	Относительное изменение
Свободный член (вход)	1101	1102	1	~0.09%
Решение x (выход)	1	-9	10	1000% (знак развернулся!)
Решение y (выход)	1	2	1	100% (выросло в два раза)